
Interactive Visualization with R: Shiny & Beyond

Instructor: Yuanyuan Wen · Virginia Tech

DATA SCIENCE FOR THE PUBLIC GOOD (DSPG) · JUNE 17, 2026

Why Interactive?

Static vs. Interactive

Static plots are great for:

- ▶ Papers, PDF reports, printed posters
- ▶ When the story is fully told by one view

Interactive plots add value when:

- ▶ Stakeholders want to explore the data themselves
- ▶ There are many subgroups, filters, or geographic areas
- ▶ Your deliverable lives on the web (HTML, dashboard)

DSPG Use Cases

Your project probably has some of these:

- ▶ County-level data → **interactive choropleth map**
- ▶ Data by group → **bar chart that updates when you change a filter**
- ▶ Trend over time by group → **hover-to-see-value line chart**
- ▶ Multiple variables → **linked scatter + map + table**

All of these are achievable today with the packages we will cover.

A Real DSPG Example

DSPG 2025 — Virginia Cost of Living Dashboard

aaec-vt.shinyapps.io/DSPG25_VA_Cost_of_Living

Three components — all built with tools you will learn today:

What you see	Package
Minimum cost table by county and family type	DT
Interactive choropleth map of Virginia	leaflet
Cost breakdown bar chart (filterable)	plotly

Two Tracks to Interactivity

	Shiny Apps	HTML Widgets
How it works	R keeps running; browser sends input → R re-computes → result returned	R runs once to produce HTML; JavaScript handles all interactivity in the browser
Pros	Full reactivity; server-side logic for large data / databases; complete control	No server needed; share as a static file; embeds in Quarto / GitHub Pages; less setup
Cons	Requires a running R process (local or hosted); more setup code	All data lives in the browser — not for large datasets; no server-side re-computation
Use this when	Filters, models, or outputs must recompute on the fly	You need hover / zoom / linked views and your data fits in a browser

Part 1: Shiny Apps

The Three Parts of Every Shiny App

```
library(shiny)

# 1. UI: what the user sees and interacts with
ui <- fluidPage(
  titlePanel("My App"),
  sidebarLayout(
    sidebarPanel(
      # inputs go here
    ),
    mainPanel(
      # outputs go here
    )
  )
)

# 2. Server: the logic that produces outputs from inputs
server <- function(input, output, session) {
  # reactive logic goes here
}

# 3. Launch the app
shinyApp(ui = ui, server = server)
```


Layout Functions — Syntax

Function	What it creates
<code>fluidPage(...)</code>	Responsive full-width page — the default container
<code>sidebarLayout(sidebarPanel(...), mainPanel(...))</code>	Classic two-column: controls left, output right
<code>fluidRow(column(width, ...), ...)</code>	Free grid — place anything in columns 1–12
<code>navbarPage("Title", tabPanel(...), ...)</code>	Multi-tab app with a top navigation bar
<code>fillPage(...)</code>	Full-viewport page — good for maps and dashboards

Start with `sidebarLayout` for any app with filters on one side and a chart on the other. Upgrade to `navbarPage` when you need multiple views.

UI Layouts — All Three in One App

The three layouts below are combined into a single `navbarPage` app so you can see them side by side.

```
library(shiny)

ui <- navbarPage("Layout Demo",

  # Tab 1 – sidebarLayout: classic two-column layout
  tabPanel("Sidebar",
    sidebarLayout(
      sidebarPanel(
        sliderInput("bins", "Number of bins:", min = 5, max = 50, value = 20)
      ),
      mainPanel(
        plotOutput("hist")
      )
    ),
  ),

  # Tab 2 – fluidRow + column: flexible grid layout
  tabPanel("Grid",
    fluidRow(
      column(4, wellPanel(
        selectInput("var", "Variable:", choices = c("mpg", "hp", "wt"))
      )),
    ),
  ),
)
```

→ Open `R/demos/shiny01/01-layout-demo.R` to explore all three layouts interactively.

Common Input Functions — Syntax

Function	Key arguments	What it creates
<code>sliderInput(id, label, min, max, value)</code>	<code>step</code> , <code>animate</code>	Numeric slider
<code>selectInput(id, label, choices, selected)</code>	<code>multiple = TRUE</code> for multi-select	Dropdown
<code>radioButtons(id, label, choices, selected)</code>	<code>inline = TRUE</code>	Radio buttons
<code>checkboxGroupInput(id, label, choices, selected)</code>	—	Check-box group
<code>textInput(id, label, value)</code>	<code>placeholder</code>	Free-text box
<code>dateRangeInput(id, label, start, end)</code>	<code>format</code>	Date range picker

Every input has an **inputId** (how you read it in the server as `input$id`) and a **label** (what the user sees).

Common Input Functions — Combined Demo

```
library(shiny); library(palmerpenguins); library(tidyverse)

ui <- navbarPage("Input Functions Demo",

  tabPanel("selectInput (single)",
    sidebarLayout(
      sidebarPanel(
        selectInput("species1", "Species:",
          choices = c("Adelie", "Chinstrap", "Gentoo"))
      ),
      mainPanel(plotOutput("plot1"))
    )
  ),

  tabPanel("selectInput (multiple)",
    sidebarLayout(
      sidebarPanel(
        selectInput("species2", "Species (pick any):",
          choices = c("Adelie", "Chinstrap", "Gentoo"),
          multiple = TRUE,
          selected = c("Adelie", "Gentoo"))
      ),
    )
  )
)
```

`req(input$x)` stops execution silently when a multi-select is empty — prevents errors on first load.

→ Open `R/demos/shiny01/02-input-functions-demo.R` to try all four input types side by side.

Output Functions — Syntax

Every output is a pair: one function in the **UI** (where to display it), one in the **server** (how to compute it).

UI function	Server function	Renders
<code>plotOutput("id")</code>	<code>renderPlot({})</code>	Static ggplot2 chart
<code>plotlyOutput("id")</code>	<code>renderPlotly({})</code>	Interactive plotly chart
<code>leafletOutput("id")</code>	<code>renderLeaflet({})</code>	Interactive leaflet map
<code>DTOutput("id")</code>	<code>renderDT({})</code>	Interactive DT table
<code>tableOutput("id")</code>	<code>renderTable({})</code>	Simple HTML table
<code>textOutput("id")</code>	<code>renderText({})</code>	Text string
<code>verbatimTextOutput("id")</code>	<code>renderPrint({})</code>	Console-style output

The `id` string must match exactly between the UI and the server: `plotOutput("scatter")` ↔ `output$scatter`.

Output Functions — Swapping Static for Interactive

The **only two lines** that change when you swap `plotly` or `leaflet` into a Shiny app:

```
# --- Static ggplot version ---
# UI:      plotOutput("chart")
# Server: output$chart <- renderPlot({ ggplot(...) })

# --- Interactive plotly version ---
# UI:      plotlyOutput("chart")
# Server: output$chart <- renderPlotly({ ggplotly(ggplot(...)) })

# --- Interactive leaflet version ---
# UI:      leafletOutput("map", height = 500)
# Server: output$map <- renderLeaflet({
#       leaflet(df) |> addTiles() |> addCircleMarkers(~lng, ~lat)
# })

# --- Interactive DT table version ---
# UI:      DTOutput("tbl")
# Server: output$tbl <- renderDT({ datatable(df) })
```

Server: `reactive()` — Shared Intermediate Values

When multiple outputs need the same filtered or transformed data, compute it once with `reactive()` and reuse it everywhere.

```
server <- function(input, output, session) {  
  
  # reactive() runs whenever input$species changes,  
  # and caches the result for all outputs that need it  
  filtered <- reactive({  
    penguins |>  
      filter(species == input$species) |>  
      drop_na()  
  })  
  
  # Both outputs call filtered() — no duplicated filter code  
  output$scatter <- renderPlot({  
    ggplot(filtered(), aes(bill_length_mm, body_mass_g)) +  
      geom_point(color = "#861F41") + theme_minimal(base_size = 13)  
  })  
  
  output$summary <- renderTable({  
    filtered() |> summarise(n = n(), mean_mass = mean(body_mass_g))  
  })  
}
```

Call a reactive object with `()` — `filtered()` not `filtered`. The result is **cached**: if two outputs call it in the same update cycle, the filter runs only once.

Demo: Virginia County Explorer

```
library(shiny); library(tidyverse); library(tidycensus)
library(leaflet); library(sf); library(scales)

vars <- c("Median Income ($)" = "B19013_001", "Median Age (years)" = "B01002_001",
         "Median Home Value ($)" = "B25077_001", "Population" = "B01003_001")

# Pull data ONCE at startup – outside ui and server
va <- get_acs(geography = "county", state = "VA", variables = vars,
             year = 2022, geometry = TRUE) |>
  st_transform(4326) |>
  mutate(county = str_remove(NAME, ", Virginia"))

ui <- fluidPage(
  titlePanel("Virginia County Explorer"),
  sidebarLayout(
    sidebarPanel(
      selectInput("county", "County:", choices = sort(unique(va$county)),
                 selected = "Fairfax County"),
      selectInput("var", "Variable:", choices = names(vars))
    ),
    mainPanel(
      textOutput("headline"),
```

→ Open `R/demos/shiny01/06-va-county-explorer.R` to run the full app.

ACS Demo: Putting It All Together

Variable → US State Choropleth

Select a variable → see it mapped across all 50 states. Uses `tidycensus` + `usmap`.

```
library(shiny); library(tidyverse); library(tidycensus); library(usmap)

vars <- c("Median Income ($)" = "B19013_001", "Median Age (years)" = "B01002_001",
         "Median Home Value ($)" = "B25077_001", "Poverty Rate (%)" = "B17001_002")

# Pull all variables for all states once at startup
state_data <- get_acs(geography = "state", variables = vars,
                     year = 2022, survey = "acs5") |>
  mutate(state = NAME)

ui <- fluidPage(
  titlePanel("US State ACS Map"),
  sidebarLayout(
    sidebarPanel(
      selectInput("map_var", "Variable:", choices = names(vars),
                 selected = "Median Income ($)")
    ),
    mainPanel(plotOutput("choro", height = "500px"))
  )
)

server <- function(input, output, session) {
```

`get_acs()` is called once outside `ui/server` so data loads at startup, not on every user interaction. `reactive()` then filters in-memory.

→ Open `R/demos/shiny01/03-acs-choropleth.R` to run the full app.

State → All-Variable Summary Table

Select a state → see a summary table of all variables for that state.

```
library(shiny); library(tidyverse)

state_df <- as.data.frame(state.x77) |>
  tibble::rownames_to_column("state") |>
  rename(
    `Per Capita Income ($)` = Income,
    `Illiteracy Rate (%)` = Illiteracy,
    `Life Expectancy (years)` = `Life Exp`,
    `Murder Rate (per 100k)` = Murder,
    `HS Graduation Rate (%)` = `HS Grad`,
    `Mean Frost Days` = Frost,
    `Population (thousands)` = Population,
    `Land Area (sq mi)` = Area
  )

ui <- fluidPage(
  titlePanel("State Profile"),
  sidebarLayout(
    sidebarPanel(
      selectInput("state_pick", "Select a state:",
                  choices = sort(state_df$state), selected = "Virginia")
    ),
  ),
```

→ Open `R/demos/shiny01/04-acs-state-summary.R` to run the full app.

Multi-Select Variables → Summary Statistics Table

Select any combination of variables → see N, mean, SD, min, median, max for each.

```
library(shiny); library(tidyverse)

state_df <- as.data.frame(state.x77) |>
  tibble::rownames_to_column("state") |>
  rename(
    income      = Income,
    illiteracy  = Illiteracy,
    life_exp    = `Life Exp`,
    murder_rate = Murder,
    hs_grad     = `HS Grad`,
    frost_days  = Frost,
    population  = Population,
    area        = Area
  )

numeric_vars <- c("income", "illiteracy", "life_exp",
                  "murder_rate", "hs_grad", "frost_days",
                  "population", "area")

ui <- fluidPage(
  titlePanel("Variable Summary Statistics"),
  sidebarLayout(
```

`req(input$var_pick)` prevents the table from crashing when the user deselects all variables.

→ Open `R/demos/shiny01/05-acs-multivar-summary.R` to run the full app.

Leaflet with Multiple Layers

Leaflet Multi-Layer: The Pattern

Add each layer as a named **group**, then use `leafletProxy()` to show or hide groups without redrawing the whole map.

```
# UI: checkbox drives which layers are visible
checkboxGroupInput("layers", "Show layers:",
               choices = c("Income", "Poverty Rate", "Median Age"),
               selected = "Income")

# Server: build the map once with all layers, two hidden by default
output$map <- renderLeaflet({
  leaflet(va) |>
    addTiles() |>
    addPolygons(group = "Income", ...) |>
    addPolygons(group = "Poverty Rate", ...) |>
    addPolygons(group = "Median Age", ...) |>
    hideGroup("Poverty Rate") |>
    hideGroup("Median Age")
})

# observe() reacts to checkbox changes and toggles groups
observe({
  proxy <- leafletProxy("map", session)
  for (layer in c("Income", "Poverty Rate", "Median Age")) {
    if (layer %in% input$layers) proxy |> showGroup(layer)
    else proxy |> hideGroup(layer)
  }
})
```

`leafletProxy()` updates an existing map in-place — the viewport and zoom level are preserved. Use it for any server-side map update after first render.

Demo: VA County Layer Explorer

Three `checkboxGroupInput`-controlled layers over Virginia counties, each with its own palette and legend.

```
library(shiny); library(tidyverse); library(tidycensus); library(leaflet); library(sf); library(scales)

va <- get_acs(geography = "county", state = "VA",
             variables = c(income = "B19013_001", poor = "B17001_002",
                           total = "B17001_001", age = "B01002_001"),
             year = 2022, geometry = TRUE, output = "wide") |>
  st_transform(4326) |>
  mutate(county = str_remove(NAME, ", Virginia"),
         income = incomeE,
         poverty_rate = round(poorE / totalE * 100, 1),
         median_age = ageE)

pal_income <- colorNumeric(c("#FFFFFF", "#861F41"), va$income)
pal_poverty <- colorNumeric("YlOrRd", va$poverty_rate)
pal_age <- colorNumeric("Blues", va$median_age)
all_layers <- c("Median Income", "Poverty Rate", "Median Age")

ui <- fluidPage(titlePanel("Virginia County Layer Explorer"),
  sidebarLayout(
    sidebarPanel(checkboxGroupInput("layers", "Show layers:",
                                  choices = all_layers, selected = "Median Income")),
    mainPanel(leafletOutput("map", height = 560))
```

→ Open `R/demos/shiny01/07-leaflet-layers-demo.R` to run the full app.

Polishing Your Shiny App

Small touches make a big difference. Use `bslib` for a modern look:

```
library(shiny)
library(bslib)

ui <- page_sidebar(
  title = "Penguin Explorer",
  theme = bs_theme(
    bootswatch = "flatly",      # modern Bootstrap 5 theme
    primary    = "#861F41",    # VT Maroon as the accent color
    base_font  = font_google("Inter")
  ),
  sidebar = sidebar(
    sliderInput("mass_range", "Body Mass (g):",
      min = 2700, max = 6300,
      value = c(2700, 6300), step = 100),
    selectInput("species_select", "Species:",
      choices = c("All", levels(penguins$species)),
      selected = "All")
  ),
  card(
    card_header("Penguin Scatter"),
    plotOutput("scatter")
  )
)
```

`bslib` replaces `fluidPage` — same server logic, much nicer look. Swap `"flatly"` for `"cosmo"`, `"lux"`, or `"minty"` for a different style.

→ Open `R/demos/shiny01/12-polish-demo.R` to run the full app with `bslib` theming.

Part 2: HTML Widgets

Interactive Figures with `plotly`

ggplotly() — One Line to Interactive

Take any ggplot2 chart and wrap it in `ggplotly()`:

```
library(plotly)

p <- penguins |>
  drop_na() |>
  ggplot(aes(x = bill_length_mm, y = body_mass_g,
             color = species, shape = species)) +
  geom_point(alpha = 0.7, size = 2) +
  labs(title = "Penguin Bill Length vs Body Mass",
       x = "Bill Length (mm)", y = "Body Mass (g)") +
  theme_minimal(base_size = 13)

ggplotly(p)
```

Built-in Interactive Features

Just by calling `ggplotly()`, users can:

- ▶ **Hover** over any point to see values
- ▶ **Zoom** by clicking and dragging a box
- ▶ **Pan** after zooming with the pan tool
- ▶ **Click legend items** to show/hide species
- ▶ **Double-click** to reset the view
- ▶ **Download** as PNG via the toolbar

No extra code required.

Customizing Hover Tooltips

Control exactly what appears in the tooltip using the `text` aesthetic:

```
p2 <- penguins |>
  drop_na() |>
  ggplot(aes(
    x = bill_length_mm, y = body_mass_g,
    color = species,
    text = paste0(
      "Species: ", species, "<br>",
      "Bill: ", bill_length_mm, " mm<br>",
      "Mass: ", body_mass_g, " g<br>",
      "Island: ", island
    )
  )) +
  geom_point(alpha = 0.7, size = 2) +
  theme_minimal(base_size = 13)

ggplotly(p2, tooltip = "text")
```

Native `plot_ly()` — Bar Chart Example

For full control, build charts natively in plotly:

```
penguins |>
  drop_na() |>
  group_by(species) |>
  summarise(mean_mass = mean(body_mass_g)) |>
  plot_ly(
    x = ~species,
    y = ~mean_mass,
    type = "bar",
    color = ~species,
    colors = c("#861F41", "#E5751F", "#2D2D2D")
  ) |>
  layout(
    title = "Average Body Mass by Species",
    xaxis = list(title = "Species"),
    yaxis = list(title = "Mean Body Mass (g)"),
    showlegend = FALSE
  )
```

Your Turn: Make a Plot Interactive

Task (5 minutes):

1. 1 Pick any ggplot2 chart you built in Workshop 1
2. 2 Assign it to a variable `p`
3. 3 Call `ggplotly(p)` — that is it!
4. 4 **Bonus:** Add a custom `text` aesthetic and use `tooltip = "text"`

```
p <- ggplot(your_data, aes(...)) +  
  geom_point() +  
  theme_minimal(base_size = 13)  
  
ggplotly(p)
```

Saving and Sharing HTML Widgets

Any widget (`plotly`, `leaflet`, `DT`, `crosstalk`) can be saved as a standalone HTML file:

```
library(htmlwidgets)

# Save any widget to a self-contained HTML file
p <- ggplotly(my_ggplot)
saveWidget(p, "my_chart.html", selfcontained = TRUE)

m <- leaflet(data) |> addTiles() |> addCircleMarkers(...)
saveWidget(m, "my_map.html", selfcontained = TRUE)
```

Ways to share:

- ▶ **Email** the `.html` file directly — opens in any browser, no R needed
- ▶ **Google Drive / Dropbox** — upload and share the link
- ▶ **Quarto report** — embed widgets directly; render `.qmd` → HTML and they are included automatically
- ▶ **GitHub Pages** — commit the HTML file and share the public URL

`selfcontained = TRUE` bundles all JavaScript into one file (~5–20 MB). For lighter files, use `selfcontained = FALSE` — R creates a companion `_files/` folder alongside the HTML; share both together.

Interactive Maps with **leaflet**

leaflet() Basics

Three steps to every leaflet map:

```
library(leaflet)

leaflet() |>
  addTiles() |>                                # OpenStreetMap base tiles
  setView(lng = -80.42, lat = 37.23,           # Blacksburg, VA
          zoom = 12)
```

Adding Markers with Popups

```
capitals <- tibble(
  city = c("Richmond, VA", "Raleigh, NC", "Columbus, OH",
           "Nashville, TN", "Frankfort, KY", "Charleston, WV"),
  lat  = c(37.54, 35.78, 39.96, 36.16, 38.20, 38.35),
  lng  = c(-77.43, -78.64, -82.99, -86.78, -84.87, -81.63),
  pop  = c(226610, 467665, 905748, 689447, 28602, 48864)
)

leaflet(capitals) |>
  addTiles() |>
  setView(lng = -82, lat = 37.5, zoom = 6) |>
  addCircleMarkers(
    lng = ~lng, lat = ~lat,
    radius = ~sqrt(pop / 5000),
    color = "#861F41", fillColor = "#E5751F", fillOpacity = 0.7,
    popup = ~paste0("<b>", city, "</b><br>Population: ",
                    scales::comma(pop))
  )
```

Choropleth Maps with leaflet + tigris

Pull county boundaries directly from the Census API with `tigris`:

```
library(tigris)
library(sf)
library(tidycensus)

va_counties <- counties(state = "VA", cb = TRUE, year = 2022) |>
  st_transform(crs = 4326)

va_acs <- get_acs(geography = "county", state = "VA",
  variables = "B19013_001", year = 2022) |>
  select(GEOID, estimate)

va_map <- left_join(va_counties, va_acs, by = "GEOID")

pal <- colorNumeric(palette = c("#FFFFFF", "#861F41"),
  domain = va_map$estimate)

leaflet(va_map) |>
  addTiles() |>
  addPolygons(fillColor = ~pal(estimate), fillOpacity = 0.7,
    color = "#2D2D2D", weight = 1,
    popup = ~paste0("<b>", NAME, "</b><br>Median Income: $",
      scales::comma(estimate))) |>
```

Interactive Tables with DT

DT::datatable() — Instant Sortable Table

```
library(DT)

flights |>
  slice_sample(n = 500) |>
  select(year, month, day, carrier, origin, dest,
         dep_delay, arr_delay, air_time, distance) |>
  datatable()
```

Key datatable() Options

```
flights |>
  slice_sample(n = 500) |>
  select(carrier, origin, dest, dep_delay, arr_delay, air_time) |>
  datatable(
    filter    = "top",          # per-column filter boxes
    rownames  = FALSE,
    options   = list(
      pageLength = 8,          # rows per page
      scrollX     = TRUE,       # horizontal scroll
      dom        = "Bfrtip"    # show Buttons, filter, etc.
    ),
    extensions = "Buttons",
    caption = "Sample of 500 NYC Flights (2013)"
  )
```

County Data Tables with DT

- ▶ Display **county-level data** — stakeholders can search their own community
- ▶ Add **column formatting** with `formatCurrency()`, `formatPercentage()`, `formatRound()`
- ▶ Export to CSV or Excel with `extensions = "Buttons" + buttons = c("csv", "excel")`

```
my_county_data |>
  datatable(
    filter = "top",
    extensions = "Buttons",
    options = list(
      dom = "Bfrtip",
      buttons = c("csv", "excel"),
      pageLength = 15
    )
  ) |>
  formatCurrency("median_income", currency = "$", digits = 0) |>
  formatPercentage("poverty_rate", digits = 1)
```


Your Turn: ACS Filterable Table

Build a filterable DT table of Virginia county data — open the exercise file and fill in the blanks.

→ Open [R/demos/shiny01/exercise-dt-acs-table.R](#) and fill in the blanks to build your table.

Linked Widgets with **crosstalk**

The crosstalk Idea

One `SharedData` object → all widgets see the same selection state.

```
library(crosstalk)

shared <- SharedData$new(my_data)

# Now pass `shared` instead of `my_data` to each widget
plot_ly(shared, ...)
datatable(shared, ...)
leaflet(shared) |> addCircleMarkers(...)
```

Filtering in any one widget instantly updates the others — **no server, no Shiny.**

Crosstalk Demo: Chart + Table

→ Open [R/demos/shiny01/13-crosstalk-demo.R](#) to run this live — click bars to highlight table rows.

```
cities <- tibble(
  city      = c("New York", "Los Angeles", "Chicago", "Houston",
               "Phoenix", "Philadelphia", "San Antonio", "San Diego",
               "Dallas", "San Jose"),
  state     = c("NY", "CA", "IL", "TX", "AZ", "PA", "TX", "CA", "TX", "CA"),
  lat       = c(40.71, 34.05, 41.88, 29.76, 33.45,
               39.95, 29.42, 32.72, 32.78, 37.34),
  lng       = c(-74.01, -118.24, -87.63, -95.37, -112.07,
               -75.16, -98.49, -117.16, -96.80, -121.89),
  population = c(8336817, 3979576, 2693976, 2320268, 1680992,
               1584064, 1547253, 1423851, 1304379, 1013240),
  median_inc = c(63998, 65290, 58247, 52338, 57459,
               45927, 52455, 79868, 54747, 117477)
)

shared_cities <- SharedData$new(cities)

bscols(
  widths = c(6, 6),
  plot_ly(shared_cities, x = ~city, y = ~median_inc, type = "bar",
           color = ~state) |>
  layout(title = "Median Income by City",
```

Click a bar to highlight the matching row in the table — no server needed.

Crosstalk + Leaflet Map

```
shared2 <- SharedData$new(cities)

pal_ct <- colorFactor(
  palette = c("#861F41", "#E5751F", "#2D2D2D", "#4A90D9", "#5BAD6F"),
  domain = cities$state
)

bscols(
  widths = c(7, 5),
  leaflet(shared2) |>
    addTiles() |>
    setView(lng = -98, lat = 38, zoom = 4) |>
    addCircleMarkers(
      lng = ~lng, lat = ~lat,
      color = ~pal_ct(state), fillOpacity = 0.8, radius = 8,
      popup = ~paste0("<b>", city, "</b><br>Pop: ",
                      scales::comma(population))
    ),
  datatable(shared2, rownames = FALSE,
    options = list(pageLength = 5, dom = "tp"))
)
```

Click a row in the table to highlight the city on the map.

When to Use `crosstalk`

Use `crosstalk` when:

- ▶ Your deliverable is a **static HTML file** (no server)
- ▶ You want linked filtering across 2–3 widgets
- ▶ The dataset is **small to medium** (all data goes to the browser)

Reach for Shiny when:

- ▶ You need server-side filtering (large data)
- ▶ You need dynamic data loading or database queries
- ▶ You need complex conditional logic (`observeEvent`, `eventReactive`)

Key Takeaways

What We Covered Today

Part 1 — Shiny Apps (server required):

- ▶ `ui + server + shinyApp()` — the minimal skeleton
- ▶ `bslib` — modern Bootstrap 5 themes with one function call
- ▶ `renderPlotly()` / `renderLeaflet()` — drop any widget into Shiny
- ▶ Reactivity: `input$x` triggers re-computation automatically

Part 2 — HTML Widgets (no server):

- ▶ `plotly` — `ggplotly()` turns any ggplot interactive in one line
- ▶ `ggiraph` — interactive ggplot2 geoms with drop-in replacements
- ▶ `DT` — sortable, filterable, downloadable data tables
- ▶ `leaflet` + `tigris` — interactive maps with Census-pulled boundaries
- ▶ `crosstalk` — `SharedData$new()` links all of the above, no server needed

The Decision Tree

Does your deliverable need server-side logic
(large data, database, complex conditions)?

— YES → Build a Shiny app (Part 1)

— NO → Do you need widgets to talk to each other?

— YES → Use crosstalk (SharedData)

— NO → Use standalone widgets
(plotly / leaflet / DT / ggiraph)

Resources

Start here — most useful for your DSPG project:

📖 **Mastering Shiny** (free book): mastering-shiny.org — the definitive reference, covers everything from basics to deployment

📄 **Shiny cheatsheet**: posit.co/resources/cheatsheets — all key functions on two pages; print it out

Package docs:

- ▶ **plotly**: plotly.com/r
- ▶ **leaflet** for R: rstudio.github.io/leaflet
- ▶ **DT**: rstudio.github.io/DT
- ▶ **ggiraph**: davidgohele.github.io/ggiraph
- ▶ **crosstalk**: rstudio.github.io/crosstalk
- ▶ **bslib** themes: rstudio.github.io/bslib